The Dart Language

A language made by Google in 2011 meant to supplant Javascript. Conservative in nature.

- strongly typed language
- support asynchronous programming
- uses both JIT and AOT
- Since Dart 3 it is now soundly null safe

Examples ran in DartPad.dev

```
void main() {
  variablePlayground();

}
void variablePlayground() {
 basicTypes();
}
void basicTypes() {
 int four = 4;
 double pi = 3.14;
 num someNumber = 24601;
 bool yes = true;
 bool no = false;
 int? nothing;
 print(four);
 print(pi);
 print(someNumber);
 print(yes);
 print(no);
 print(nothing == null);
}
```

The above outputs...

```
4
3.14
```

```
24601
true
false
true
```

Dynamic types are sort of a 'get out of jail free card' that can be anything and should **NOT** be used.

```
void untypedVariables() {
 dynamic something = 14.2;
 print(something.runtimeType); //outputs 'double'
}
```

Similar type inferring can be done with `var` but NOT exactly the same

```
void typeInterpolation() {
 var anInteger = 15;
 var aDouble = 27.6;
 var aBoolean = false;
 print(anInteger.runtimeType);
 print(anInteger);
 print(aDouble.runtimeType);
 print(aDouble);
 print(aBoolean.runtimeType);
 print(aBoolean);
}
```

outputs

```
int
15
double
27.6
bool
false
```

In the above snippet the variable `anInteger` is only mutable within the bounds of the `int` type.

Immutable variables

`final` and `const` must be determined at compile time. For example, a `const` variable cannot be have the value of `DateTime.now()` since the date can only determined at runtime.
`final` variables have a little leeway. Though, they can only be assigned once, if the `final` variable is assigned to an object then the value of the variable can change by adjusting the fields of that object. `const` does not have this flexibility.
Generally, it is a good idea to favor immutable over mutable because mutability is more likely to lead to errors and immutable variables are faster.

Strings and string interpolation

```
final age = 42;
final howOld = 'I am $age ${age == 1 ? 'year' : 'years'} old.';
print(howOld);
```

In the snippet above, placing simple variables in a string is done with `$` but the brackets are necessary if you want an expression to be computed in the string.

```
final rawString = r'I want to show an escape \ character';
```

The `r'..'` raw string is used to print escape characters.
Dart also has a `StringBuffer` which is like Java's string builder. The functionality is an incremental string concatenator. `StringBuffer` as better performance than normal string interpolation.

```
List fruits = ['Strawberry', 'Coconut', 'Orange', 'Mango', 'Apple'];
StringBuffer buffer = StringBuffer();
for (String fruit in fruits) { buffer.write(fruit);
        buffer.write(' ');
         } print (buffer.toString());
          // prints: Strawberry Coconut Orange Mango Apple
```

Functions

Normal structure

```
int factorial(int number){
        if (number <= 0){
        return 1;
```

```
        }
        return number * factorial(number - 1)
}
```

Optional positional parameters are wrapped in square brackets. The `?` after the type tells the Dart compiler that the parameter itself can be null.

```
void unnamed([String? name, int? age]) {
 final actualName = name ?? 'Unknown';
 final actualAge = age ?? 0;
 print('$actualName is $actualAge years old.');
}
```

In the above snippet, the double `??` is the **null-aware coalescing operator** which is a quick to check if a value is null then provide a default value if it is.
Named Optional Parameters with curly brackets `{}`. When calling a function with named parameters you NEED to specify the parameter name but the parameter list can be in any order.

```
void named({String? greeting, String? name}) {
  final actualGreeting = greeting ?? 'Hello';
  final actualName = name ?? 'Mystery Person';
  print('$actualGreeting, $actualName!');
     }
```

BOTH optional parameters and optional named parameter support default values. If a function is called with one of these styles then `null` will not be used in preference to their default value.
In the below example, there is a mix of both **required** and **named optional**.

```
String duplicate(String name, {int times = 1}) {
  final merged = StringBuffer(name);
  for (var i = 1; i < times; i++) {
    merged.write(' $name');
     }
  return merged.toString();}
```

For completeness the above is called like `duplicate('Mikey', times: 3);`

Name Parameters make readable code

When a function does not have a return type or functional parameter the **dynamic** type is used but this should be avoided.

Functions as a variable -> Closures

Closures are known as first-class functions. As a language feature they emerged form lambda calculus. Closure allow functions to be passed as parameters to other functions. To add a closure to a function, define another function signature inside a function.

```
void callbackExample(void Function(String value) callback) {
  callback('Hello Callback');
  }

void printSomething(String value) { print(value); }
```

The above closure the function is initiated by `callbackExmaple(printSomething)`.
The variable `callback` has a value of the function `printSomething`
The above snippet is defines an inline closure. A less verbose is to the type alias called `typedef`. It makes the function signature cleaner.

```
typedef CallBack = void Functions();

void callbackExample(CallBack callback ) {
  callback('Hello Callback');
  }
```

Another example...

```
    typedef NumberGetter = int Function();

    int powerOfTwo(NumberGetter getter) {
     return getter() * getter();
    }
```

The above code is activated by the below snippet. Notice the *rocket notation* for single line functions.

```
int getFour() => 4;
```

```
final squared = powerOfTwo(getFour);
```

Switch Statements

```
int dayOfWeek = 7;
//String myDay = getDay(dayOfWeek);
var myDay = switch (dayOfWeek) {
        1 => 'Monday',
        2 => 'Tuesday',
        3 => 'Wednesday',
        4 => 'Thursday',
    5 => 'Friday',
    6 => 'Saturday',
    7 => 'Sunday' ,
    _ => 'Invalid day' //Default value
    };
print(myDay);
```

Records

Since Dart 3, the **record expression** is used to hold custom data like ...

```
var person = (name: 'Clark', age: 42);
print (person.name);
```

In the above case, the variable `person` is the **record expression**. These are immutable set of fields.
The below functions `getPerdon` returns a **record** which is similar to a tuple.

```
void main() {
  var (String name, int age) = getPerson({'name': 'Clark', 'age':
42});
  print('$name is $age years old.');
   }
(String, int) getPerson(Map<String, dynamic> json) {
  return (json['name'] as String, json['age'] as int);
   }
```

Classes

Not much different from other Object Oriented languages. Classes are used to make custom types.

```
class Name {
   final String first;
   final String last;
   Name(this.first, this.last);
   @override
   String toString() {
   return '$first $last';
   }
}
```

Now using the above and class through inheritance.

```
class OfficialName extends Name {
// Private properties begin with an underscore
  final String _title;
   // You can add colons after constructor
   // to parse data or delegate to super
  OfficialName(this._title, String first, String last)
          : super(first, last);
  @override
  String toString() {
    return '$_title. ${super.toString()}'; } }
```

How the above is used

```
final name = OfficialName('Mr', 'Clark', 'Kent');
final message = name.toString();
print(message)
```

A noted about the *constructor shorthand*

`const Name(this.first, this.last) : super(first, last):`

This allows member variable assignment by using `this` with dot notation.

3 keywords for building relationships

`extends` adds functionality to the super class. 1 extends per class

`implements` **all classes are implicit interfaces**. Does not inherit any code just a contract to write a the same function signatures. Infinite implements can be applied

to a class.
`with` uses `mixin`s for multiple class hierarchies. When defining a `mixin` `extends` cannot be used. A `mixin` declaration defines a `mixin`. A `class` declaration defines a class. A `mixin class` declaration defines a class that is usable as both a regular class and a `mixin`, with the same name and the same type.

Spread Operator

A brief way to add multiple values into a collection.

```
var list = [1, 2, 3];
var list2 = [0, ...list];
assert(list2.length == 4);
```

If `list` variable could be **null** use the **null-aware spread operator**

```
var list2 = [0, ...?list];
```

Higher-Order Functions

Functions that take one or more functions as arguments or return a function.
Given data of this form. The goal is to turn this mapping into a strongly typed object.

```
List<Map> data = [ {'first': 'Nada', 'last': 'Mueller', 'age': 10},
                                    ...
                                    ];
```

We will use the previous seen Name class

```
class Name {
  final String first;
  final String last;
  Name(this.first, this.last);
  @override
  String toString() {
   return '$first $last'; } }
```

We will use `map` function to transform the data into a new form. From a `Map` type to encapsulating in a class (object) called `Name`

Imagine the `data` variable above is a global variable and is in scope of the function `mapping()` below.

```
List<Name> mapping() {
// Transform the data from raw maps to a strongly typed model
final names = data.map<Name>((Map rawName) {
  final first = rawName['first'];
  final last = rawName['last'];
  return Name(first, last); }).toList();
return names; }
```

The above snippet can also be down with loop below. Notice how the mapping takes care of the looping and the adding to the list.

```
final names = <Name>[];
for (Map rawName in data) {
  final first = rawName['first'];
  final last = rawName['last'];
  final name = Name(first, last);
  names.add(name); }
```

Now that the `map` function's internals are understood there is even a more concise way translate the data variable of type `Map` into a strongly typed object.

```
final names = data.map<Name>(
  (raw) => Name(raw['first'], raw['last']), ).toList();
```

Now that data is properly formatted we can sort by the object's property (last name). Notice no new variable is created after `sort` is called therefore, **in place sort**

```
void sorting() {
  final names = mapping();
   // Sort the list by last name
   names.sort((a, b) => a.last.compareTo(b.last));
   print('');
   print('Alphabetical List of Names');
   names.forEach(print); }
```

Pull a subset of data from the list of Name objects.

```
void filtering() {
  final names = mapping();
  final onlyMs = names.where((name) => name.last.startsWith('M'));
  print('');
  print('Filters name list by M');
  onlyMs.forEach(print); }
```

Calculate average age. `reduce` works like a running sum expression

```
void reducing() {
// Merge an element of the data together
final allAges = data.map<int>((person) => person['age']);
final total = allAges.reduce((total, age) => total + age);
final average = total / allAges.length;
print('The average age is $average'); }
```

Flattening a multi-dimensional list to 1 dimensional.

```
final matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [-7, -8, -9],
];
final linear = matrix.expand<int>((row) => row);
```

`(1, 2, 3, 4, 5, 6, -7, -8, -9)`

Quick print of an iterable

```
// Instead of doing this
data.forEach((value) { print(value); });
// We can do this
data.forEach(print);
```

Brevity by chaining higher-order functions

```
final names = data .map<Name>(
  (raw) => Name(raw['first'], raw['last']))
  .where((name) => name.last.startsWith('M'))
  .where((name) => name.first.length > 5)
  .toList(growable: false);
```

Iterables are *lazy* so each high-order function is cached and only ran once the `.toList()` is called. The above is a single pass algorithm. By default in dart all lists are growable. Here, we declare the list cannot get any bigger than its initial size.

First class functions versus higher-order functions

- A first-class function can be treated as a value: it can be assigned to a variable, passed as an argument, or returned as a value.
- A higher-order function takes one or more functions as arguments or returns a function as a result.
  This means that first-class functions can be treated as values, and higher-order functions take or return functions as input or output.

The Builder Pattern

used when constructors get too large. The below will incrementally build out the object with one constructor call at a time.

```
class UrlBuilder {
  String? _scheme;
  String? _host;
  String? _path;
  UrlBuilder setScheme(String value) {
    _scheme = value;
    return this; }
  UrlBuilder setHost(String value) {
    _host = value;
    return this; }
  UrlBuilder setPath(String value) {
    _path = value;
    return this; }
  String build() {
    assert(_scheme != null);
    assert(_host != null);
    assert(_path != null);
    return '$_scheme://$_host/$_path'; }
}
void main() {
  final url = UrlBuilder()
          .setScheme('https')
          .setHost('dart.dev')
          .setPath('/guides/language/language-tour#cascade-notation-
')
```

```
          .build();
          print(url); }
```

The above is verbose and unnecessary

```
class UrlBuilder {
  String scheme = '';
  String host = '';
  List<String> routes = [];
  @override
  String toString() {
    final paths = [host, if (routes != []) ...routes];
    final path = paths.join('/');
    return '$scheme://$path'; } }
```

To populate the above class use the **Cascade operator**

```
final url = UrlBuilder()
        ..scheme = 'https'
        ..host = 'dart.dev'
        ..routes = [ 'guides',
                                 'language',
                                  'language-tour#cascade-notation',
];
         print(url);
```

Another Cascade example

```
final numbers = [342, 23423, 53, 232, 534];
numbers.insert(0, 10);
numbers.sort((a, b) => a.compareTo(b));
```

The above becomes this with Cascade

```
final numbers = [342, 23423, 53, 232, 534]
        ..insert(0, 10)
        ..sort((a, b) => a.compareTo(b));
```

Extensions

Extensions allow methods and properties to be adding to existing classes without modifying the original class. Especially useful when the class cannot be changed...

```
extension StringExtensions on String {
  bool toBool() { return isNotEmpty; }
  }
```

Sound Null Safety

Generally, variables that have no value are `null` but with `null` safety *variables cannot be assigned to a null value* BUT the use of `null` can be beneficial so now `null` values have to be explicitly allowed.
To force a *Uncaught TypeError* use the `!`

```
void main() {
 int? someNumber;
 increaseValue(someNumber!);
}
void increaseValue(int value) {
 value++;
 print (value);
}
```